

Lab Guide 01: Running NVIDIA NIM and GenAI-Perf Benchmark

This guide will walk you through setting up a GPT-OSS 20B model on NIM and running a GenAI-Perf benchmark using Triton Server. You'll learn how to configure the environment, launch the container, inference with the NVIDIA NIM container.

Prerequisites

Before starting, ensure you have the following:

- **Docker** (with NVIDIA Container Toolkit installed)
 - **NGC API key** (for accessing NVIDIA's NGC registry)
 - **GPU-enabled system**
-

1. Running the NIM Model

Step 1: Set Up Local Nim Cache

First, set up a local cache directory for NIM.

```
export LOCAL_NIM_CACHE=~/.cache/nim
mkdir -p "$LOCAL_NIM_CACHE"
```

Step 2: List Available Model Profiles

You can list the available model profiles for the desired model using the following command.

```
docker run --rm --runtime=nvidia --gpus=all \
-e NGC_API_KEY=$NGC_API_KEY \
```

```
nvcr.io/nim/openai/gpt-oss-20b:2.0.6 \  
list-model-profiles
```

Step 3: Start the Model Server

Run the GPT-OSS-20B Model on NIM in detached mode.

```
docker run -itd --name=gpt-oss-20b --rm \  
  --gpus all \  
  --shm-size=16GB \  
  -e NGC_API_KEY \  
  -v "$LOCAL_NIM_CACHE:/opt/nim/.cache" \  
  -u $(id -u) \  
  -p 8000:8000 \  
  nvcr.io/nim/openai/gpt-oss-20b:2.0.6
```

It will take a while for the model to be deployed (around 1~5 mins). Run the following command to continuously check for GPU utilization, and you should see something like the following:

```
watch nvidia-smi
```

```
> nvidia-smi
Thu Jun 25 15:19:41 2026

+-----+
| NVIDIA-SMI 580.105.08                Driver Version: 580.105.08      CUDA Version: 13.0     |
+-----+-----+
| GPU  Name           Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp    Perf       Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |
+-----+-----+
|  0  NVIDIA RTX Pro 6000 Blac...    On | 00000000:02:00.0 Off |           0         |
| N/A   N/A     P0              N/A / N/A | 74479MiB / 98304MiB |    0%    Default   |
|                                           | N/A              |
+-----+-----+

+-----+
| Processes: |
| GPU   GI   CI        PID   Type   Process name                      GPU Memory |
|   ID   ID   ID             |                 |           Usage   |
+-----+-----+
|  0     N/A N/A        2169    G   /usr/lib/xorg/Xorg                  67MiB |
|  0     N/A N/A       96554    C   VLLM::EngineCore                   74401MiB |
+-----+-----+
```

Press **CTRL + C** to go back and proceed to step 4.

Step 4: Test the Model Endpoint

To verify that the model server is running, send a **GET** request to list the available models.

```
curl -s -X GET 'http://0.0.0.0:8000/v1/models' | jq
```

```
> curl -s -X GET 'http://0.0.0.0:8000/v1/models' | jq
{
  "object": "list",
  "data": [
    {
      "id": "openai/gpt-oss-20b",
      "object": "model",
      "created": 1782397075,
      "owned_by": "vllm",
      "root": "/opt/nim/.cache/tmp/nim_q4ps6ky6",
      "parent": null,
      "max_model_len": 131072,
      "permission": [
        {
          "id": "modelperm-852c08aae73e4de7",
          "object": "model_permission",
          "created": 1782397075,
          "allow_create_engine": false,
          "allow_sampling": true,
          "allow_logprobs": true,
          "allow_search_indices": false,
          "allow_view": true,
          "allow_fine_tuning": false,
          "organization": "*",
          "group": null,
          "is_blocking": false
        }
      ]
    }
  ]
}
```

Step 5: Test Model Response

You can now test the model by sending a POST request with a sample input.

```
curl -X 'POST' \
'http://0.0.0.0:8000/v1/chat/completions' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
```

```
-d '{
  "model": "openai/gpt-oss-20b",
  "messages": [{"role": "user", "content": "Which number is larger, 9.11 or 9.8?"}],
  "max_tokens": 1024
}' | jq
```

You should see the following response, the LLM response is under the `content` field:

```
> curl -X 'POST' \
'http://0.0.0.0:8000/v1/chat/completions' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "model": "openai/gpt-oss-20b",
  "messages": [{"role": "user", "content": "Which number is larger, 9.11 or 9.8?"}],
  "max_tokens": 1024
}' | jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
100    1174    100    1028    100     146    1627     231   --:--:-- --:--:-- --:--:--   1857
{
  "id": "chatcmpl-97167ddd7fecf8c8",
  "object": "chat.completion",
  "created": 1782440415,
  "prompt_routed_experts": null,
  "model": "openai/gpt-oss-20b",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "9.8 is larger. \n- 9.8 is almost 10, whereas 9.11 is just over 9. \n- In terms of place value, 9.8 has a higher tenths digit (8 vs 1). \nSo 9.8 > 9.11.",
        "refusal": null,
        "annotations": null,
        "audio": null,
        "function_call": null,
        "tool_calls": [],
        "reasoning": "The user asks: Which number is larger, 9.11 or 9.8? It's straightforward: 9.8 > 9.11, because 9.8 is closer to 10 and 9.11 ~ 9.11. The difference: 9.8 is 0.9 larger. The answer: 9.8 is larger. Provide explanation."
      },
      "logprobs": null,
      "finish_reason": "stop",
      "stop_reason": null,
      "token_ids": null,
      "routed_experts": null
    }
  ],
  "service_tier": null,
  "system_fingerprint": "vllm-0.21.0-c091cf68",
  "usage": {
    "prompt_tokens": 82,
    "total_tokens": 236,
    "completion_tokens": 154,
    "prompt_tokens_details": null
  },
  "prompt_logprobs": null,
  "prompt_token_ids": null,
  "prompt_text": null,
  "kv_transfer_params": null
}
```

2. Running GenAI-Perf Benchmark

Step 6: Copy the Tokenizer for GPT-OSS-20B

Set the tokenizer for the GPT-OSS-20B model.

```
export HF_TOKENIZER=~/tokenizer
```

Step 7: Export Variables and Run Triton Server

Set the environment variables and run the Triton Server.

```
export RELEASE="24.06" # Use the latest releases in yy.mm format
export WORKDIR=~/genai-perf
mkdir -p "$WORKDIR"
docker run -it --rm --net=host --gpus=all \
  -v $WORKDIR:/workdir \
  -v $HF_TOKENIZER:/root/.cache/huggingface \
  nvcr.io/nvidia/tritonserver:${RELEASE}-py3-sdk
```

Step 8: Run GenAI-Perf Benchmark

Run the GenAI-Perf benchmark script on the Triton Server. Allow approximately 30 seconds for the script to complete.

```
export INPUT_SEQUENCE_LENGTH=200
export INPUT_SEQUENCE_STD=10
export OUTPUT_SEQUENCE_LENGTH=200
export CONCURRENCY=10
export MODEL=openai/gpt-oss-20b

cd /workdir
genai-perf \
  -m $MODEL \
  --endpoint-type chat \
  --service-kind openai \
  --streaming \
  -u localhost:8000 \
  --synthetic-input-tokens-mean $INPUT_SEQUENCE_LENGTH \
  --synthetic-input-tokens-stddev $INPUT_SEQUENCE_STD \
```

```
--concurrency $CONCURRENCY \
--output-tokens-mean $OUTPUT_SEQUENCE_LENGTH \
--extra-inputs max_tokens:$OUTPUT_SEQUENCE_LENGTH \
--extra-inputs min_tokens:$OUTPUT_SEQUENCE_LENGTH \
--extra-inputs ignore_eos:true \
--tokenizer openai/gpt-oss-20b \
-- \
-v \
--max-threads=256
```

The benchmark can take up to few minutes, in which once done you will see the following reference benchmark output:

LLM Metrics

Statistic	avg	min	max	p99	p90	p75
Time to first token (ns)	1,278,402,717	512,195,421	1,596,879,556	1,587,990,753	1,538,258,744	1,499,868,829
Inter token latency (ns)	8,044,127	6,643,543	8,948,303	8,924,418	8,609,350	8,438,243
Request latency (ns)	1,566,103,638	1,513,970,193	1,665,104,465	1,665,081,438	1,634,004,615	1,582,216,588
Num output token	36	1	133	131	77	45
Num input token	199	180	219	218	211	206

Output token throughput (per sec): 35.54
Request throughput (per sec): 6.68
root@ubuntu-24-04:/workdir#

Step 9: Exit out of container

Exit out of the current triton container:

```
exit
```

Summary

Congratulations! 🎉 You successfully launched the OpenAI GPT-OSS 20B model on NIM, tested it with sample queries, set up Triton Server, and ran a GenAI-Perf benchmark to measure performance